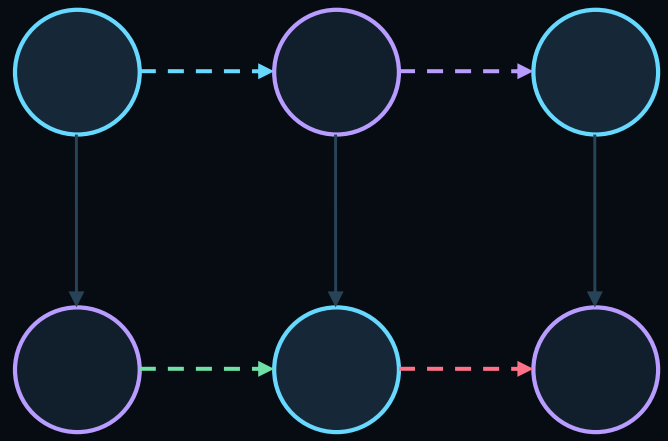
VERITAS

An Evidence-Aware, Temporal and Explainable Database for Evolving AI Debates

From answering questions to understanding how positions evolve.

MONGODB FASTAPI REACT AI-ASSISTED



CLAIMS → EVIDENCE → SYNTHESIS

A debate is more than an answer.

Vellore Institute of Technology • BCSE406L — NoSQL Databases

Project demonstration | Fall 2026–27 | Faculty: Dr. S. Geetha

Team: 23BCE1683 • 23BCE1665 • 23BCE5048

ROADMAP

What we will demonstrate

A system walkthrough, not just a list of features.

01 **Problem & proposed contribution**

Why debates need persistent structure

02 **Architecture & MongoDB model**

How UI, API and documents connect

03 **Reasoning pipeline**

Scoring, contradictions and history

04 **Application features**

Dashboard, workspace, graph, search

05 **Database engineering**

CRUD, queries, indexes and aggregations

06 **Live demonstration**

Health → workspace → query → admin

The problem: answers disappear; reasoning is hard to inspect

A fluent response does not automatically preserve its evidence structure.

Typical chatbot flow

Question
↓
Generated answer
↓
Conversation ends

Difficult to inspect later:
- claim-to-evidence links
- provenance
- changed assumptions
- historical confidence

What researchers need

Question
↓
Claims and positions
↓
Evidence and counterarguments
↓
Conflicts and uncertainty
↓
New evidence / revised state

VERITAS approach

Store the debate as a durable data object.

Each part is queryable and connected.

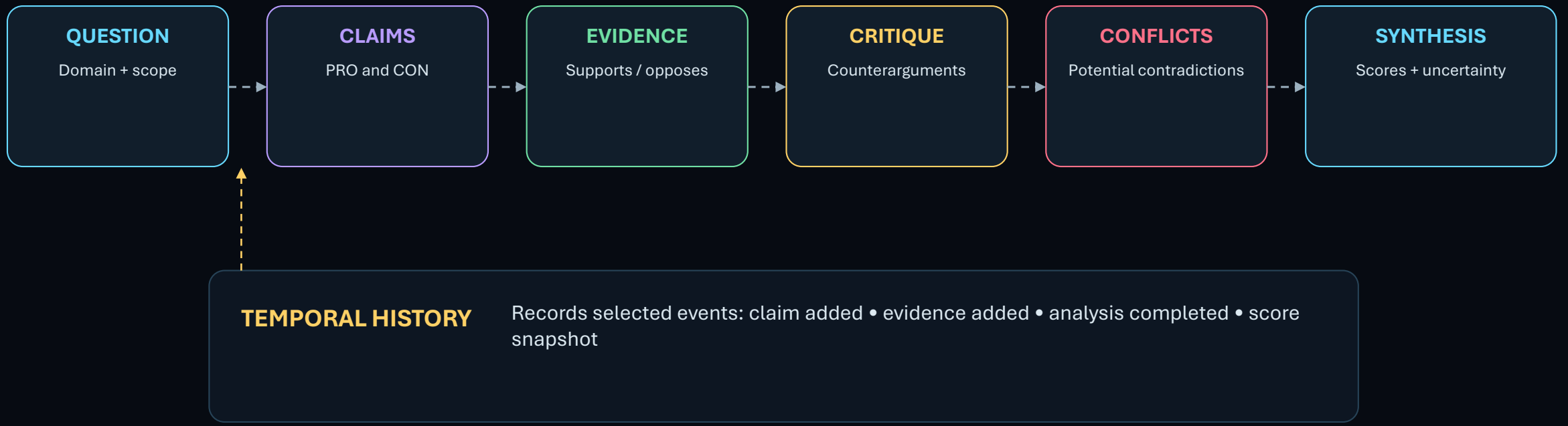
Selected state changes are kept in history.

The system explains its inputs; it does not prove objective truth.

Core question: how can competing arguments be stored, connected, inspected and updated over time?

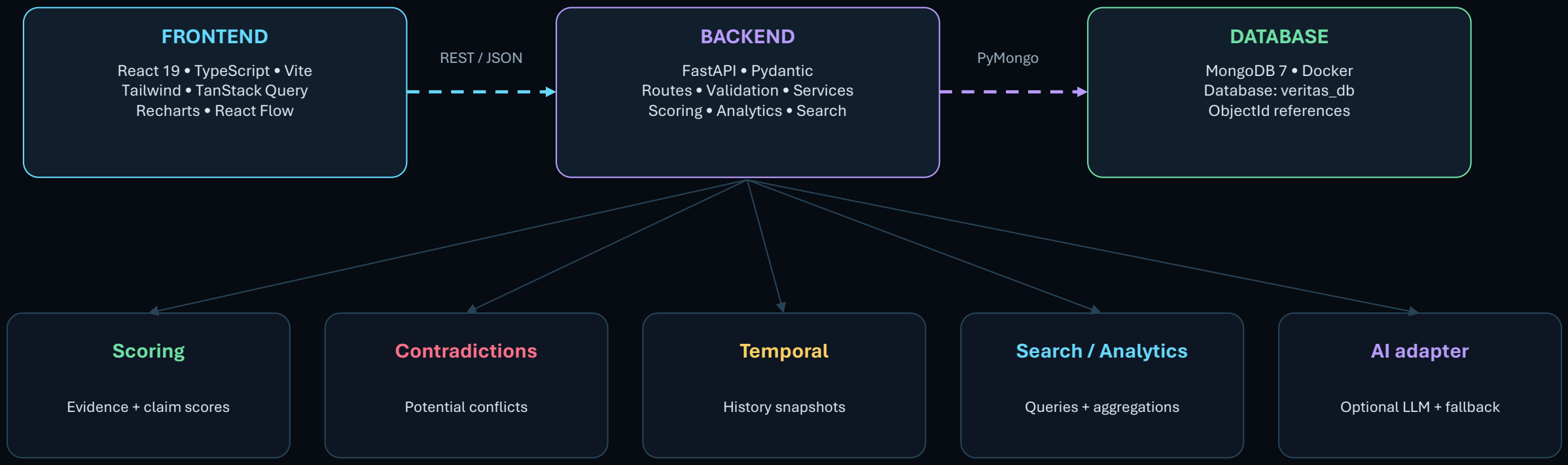
From one question to an evolving knowledge object

The debate lifecycle is the central abstraction.



Implemented architecture: three main layers plus services

React requests → FastAPI workflows → PyMongo → MongoDB.



LOCAL DEVELOPMENT MongoDB may be unavailable while the API process remains up; data endpoints then report errors or degraded health.

Technology stack and why each part is used

Choose tools that support the project requirements without unnecessary infrastructure.

React + TypeScript	Component UI, typed frontend data
Vite + Tailwind CSS	Fast development server and consistent styling
TanStack Query + Router	Fetch/cache server data and manage navigation
Recharts + React Flow	Timeline / KPI charts and argument graph
FastAPI + Pydantic	REST API, schema validation, typed requests
PyMongo + MongoDB 7	Document reads/writes, references, aggregation
OpenAI-compatible adapter	Optional structured synthesis generation
Deterministic fallback	Keep local demo usable without an LLM key

Intentionally not used: Kubernetes, Kafka, Redis clusters or microservice sprawl.

MongoDB data model: nine intended collections

Eight collections were tracked by the latest runtime health check; audit_logs is intended but may be empty.

debates

Question, domain, status, metadata and embedded current_synthesis

claims

PRO/CON text, confidence, tags and debate reference

evidence

Support/opposition, source attributes, scores and provenance

counterarguments

Challenges and qualifications linked to a claim

contradictions

Potential conflicts between claim A and claim B

sources

Publisher, type, URL, publication date and reliability

debate_history

Timestamped state snapshots and event summaries

embeddings

Vectors associated with claims/evidence for search experiments

audit_logs

Intended audit trail; active write coverage not assumed

What a document looks like

ObjectIds keep records connected while each collection has its own lifecycle.

DEBATE DOCUMENT

```
{
  _id: ObjectId(...),
  question: "Should AI be used...",
  domain: "Healthcare",
  status: "active",
  current_synthesis: {
    summary: "...",
    pro_score: 0.625,
    con_score: 0.61,
    confidence: 0.6175,
    uncertainty: 0.985
  },
  metadata: { language: "en" }
}
```

CLAIM DOCUMENT

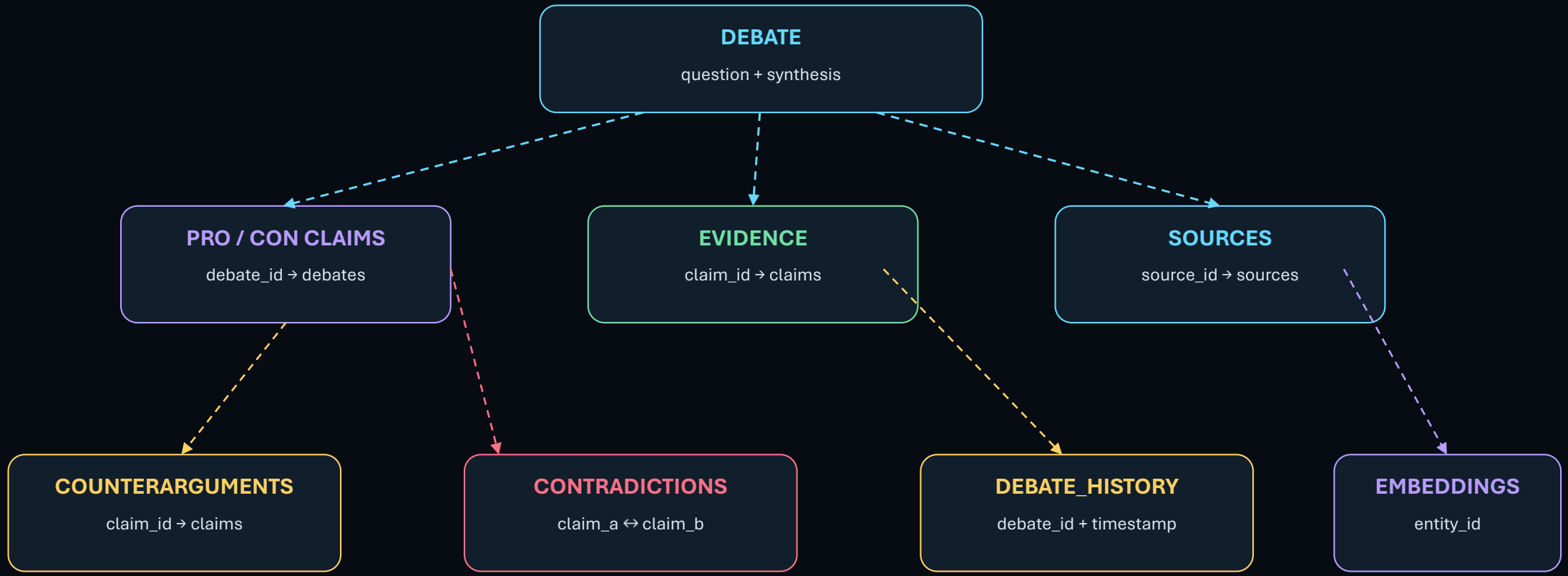
```
{
  _id: ObjectId(...),
  debate_id: ObjectId(...),
  position: "PRO",
  text: "AI can assist diagnosis...",
  confidence: 0.60,
  status: "active",
  evidence_ids: [...],
  counterargument_ids: [...],
  provenance: {
    generated_by: "retrieval"
  }
}
```

EVIDENCE DOCUMENT

```
{
  _id: ObjectId(...),
  claim_id: ObjectId(...),
  source_id: ObjectId(...),
  supports: true,
  reliability_score: 0.50,
  relevance: 0.55,
  independence_score: 0.55,
  recency_score: 0.60,
  evidence_score: 0.0908,
  url: null
}
```

The debate graph and collection relationships

VERITAS uses a graph of linked documents rather than one giant nested record.



Why embed some fields and reference other records?

The strategy balances read efficiency, duplication and independent growth.

EMBED

Embed compact, tightly coupled fields:

- `current_synthesis` inside `debates`
- compact metadata

Why?
- read with the debate header
- avoids a separate fetch for current scores
- size and lifecycle are bounded

REFERENCE

Reference independent, reusable or growing records:

- claims → debates
- evidence → claims / sources
- contradictions → two claims
- history → debate
- embeddings → entity

Why?
- independent updates
- many-to-one / cross-claim links
- avoid giant nested arrays

Design trade-off: faster common reads vs. lower duplication and clearer independent lifecycles.

End-to-end workflow: create, analyze, inspect, update

This is the principal journey to show during the live demo.

1

Create debate

Question + domain

POST /api/debates

2

Store root

Initial synthesis + event

MongoDB: debates

3

Analyze

Claims/evidence/rebuttals

POST /api/ai/analyze

4

Recalculate

Scores + conflicts

Python services

5

Persist history

Current state + snapshot

debate_history

6

Explore

Graph, query, timeline

React workspace

Important side effect Generate Analysis writes claims, fixture evidence/counterarguments, updated scores and history. Use a disposable debate.

Evidence scoring: transparent, deterministic arithmetic

The formula is implemented, but it has not been empirically validated.

Evidence Score = Reliability × Relevance × Independence × Recency

Reliability

How trustworthy is the source metadata?

Relevance

How directly is it tied to this claim?

Independence

Does it add independent evidence?

Recency

How recent is the information?

Example inputs

0.50 × 0.55 × 0.55 × 0.60

Computed example

0.0908

From evidence scores to claim and debate-level scores

The outputs summarize the current stored inputs; they are not probabilities of truth.

CLAIM STRENGTH

Starting from supporting evidence:

+ supporting contribution
− half-weighted opposing evidence
− counterargument penalty
− contradiction penalty

Normalize to 0–1.

DEBATE SCORES

PRO score = average confidence of active PRO claims

CON score = average confidence of active CON claims

Superseded claims are excluded.

UNCERTAINTY

Derived from how close the PRO and CON scores are.

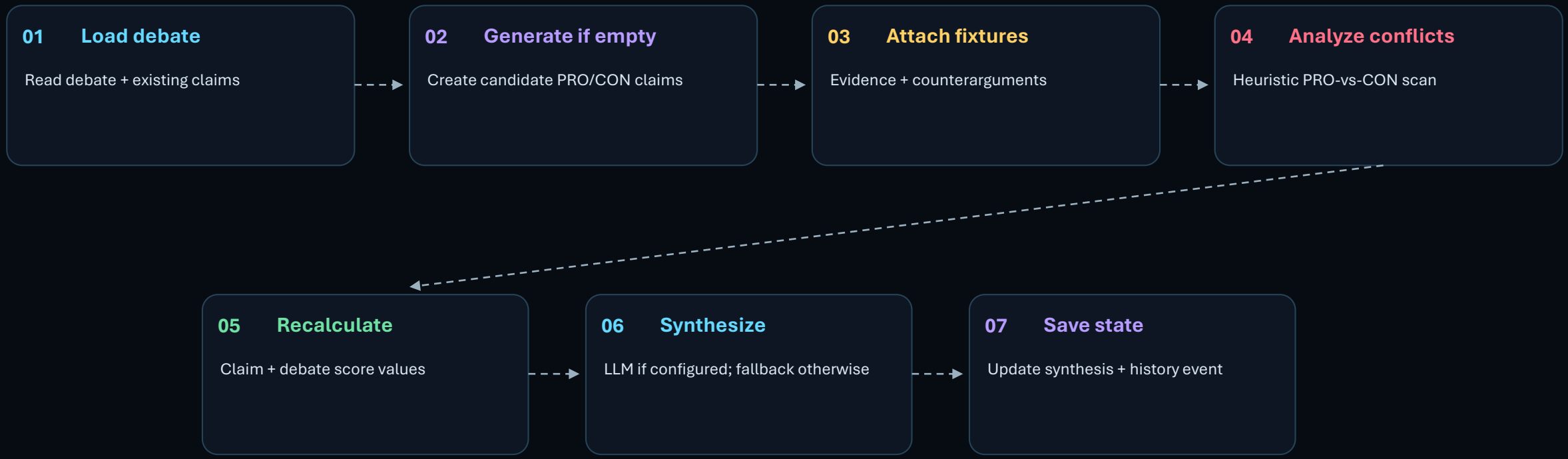
Close scores imply unresolved competition in this framework.

Not a calibrated probability.

Documented Healthcare values: PRO average 0.625 • CON average 0.610

Generate Analysis: implemented pipeline and fallback

Optional LLM synthesis; a deterministic fallback keeps local demos working.



Contradiction detection: useful flagging, not proof

The detector compares PRO claims with CON claims and records potential conflicts.

INPUT

PRO claim
“AI improves diagnosis...”

CON claim
“AI can perform worse for certain groups...”

Compare possible opposing statements.

METHOD

1. Token-set Jaccard overlap
2. Small positive/negative word-list check
3. Heuristic strength
4. Categorize and explain
5. Avoid duplicate pairs

OUTPUT TYPES

`direct_contradiction`

`contextual_disagreement`

`potential_conflict`

Edges can appear in the debate graph.

A real conflict may depend on population, geography, method, dataset, time period or assumptions.

Temporal history: preserve selected changes over time

The timeline is generated from application-level snapshots and events.



debate_created
Initial question

claim_added
New PRO/CON claim

evidence_added
Evidence + score

analysis_completed
Synthesis updated

later update
New snapshot

Stored snapshot fields timestamp • PRO score • CON score • confidence • evidence count • contradiction count • event summary

Search: keyword retrieval and semantic-search experiments

The app provides two paths, with an honest fallback when trained embeddings are unavailable.

KEYWORD SEARCH

Searches fields including:

- debate question / description / domain
- claim text
- evidence title / content
- source title

Can filter by domain and limit results.
Uses case-insensitive matching.

SEMANTIC SEARCH PATH

1. Embed query
2. Load claim/evidence vectors
3. Compute cosine similarity in Python
4. Rank candidates
5. Apply filters: source type, reliability, support/opposition

Falls back to keyword search if vectors are absent.

CRUD and API surface

The API provides controlled operations for debates, claims, evidence and related resources.

DEBATES

```
POST   /api/debates
GET    /api/debates
GET    /api/debates/{id}
PUT    /api/debates/{id}
DELETE /api/debates/{id}
```

CLAIMS

```
POST   /api/claims
GET    /api/claims/{id}
GET    /api/claims/debate/{id}
PUT    /api/claims/{id}
DELETE /api/claims/{id}
```

EVIDENCE

```
POST   /api/evidence
GET    /api/evidence/{id}
GET    /api/evidence/claim/{id}
PUT    /api/evidence/{id}
DELETE /api/evidence/{id}
```

Validation and serialization Pydantic validates payloads; MongoDB ObjectIds and Dates are converted to JSON-friendly responses.

Query Explorer: 12 predefined MongoDB operations

Safe named queries expose meaningful patterns without arbitrary browser-side Mongo execution.

RETRIEVE & FILTER

- Q1 List debates by newest
- Q2 Claims for a debate
- Q4 Strongest supporting evidence
- Q5 Strongest opposing evidence
- Q8 History in timestamp order

RANK & INSPECT

- Q6 High-strength conflicts
- Q9 Low-confidence claims
- Q10 Rank claims by confidence
- Q11 Evidence-weighted score ($lookup)

AGGREGATE

- Q3 PRO vs CON average confidence
- Q7 Reliability by source type
- Q12 Domain statistics

Indexes and aggregation pipelines

Indexes match actual access patterns; aggregation computes grouped metrics inside MongoDB.

INDEX EXAMPLES

debates: `domain + created_at`
Filter by domain and sort newest first.

claims: `debate_id + position + confidence`
Load PRO/CON panels ranked by confidence.

evidence: `claim_id + supports + reliability + relevance`
Rank support/opposition evidence.

Other indexes: contradictions, history, sources, embeddings, counterarguments.

AGGREGATION EXAMPLES

1. PRO vs CON average confidence
`$match` + `$group` + `$avg`

2. Evidence quality by source type
`$group` + `$avg` + `$sort`

3. Evidence-weighted score
`$lookup` + `$map` + `$sum` + `$cond`

Also: evidence count per claim, top claims, contradiction distribution and domain statistics.

Latest documented runtime: the app is connected and serving

These values are from the latest runtime notes provided; refresh health before the faculty demo.

CONNECTED	11	489	16
MongoDB health	**Debates returned**	**Tracked documents**	**Indexes reported**
API: /api/health	GET /api/debates	health endpoint count	live count; recheck

Running endpoints

Frontend • http://127.0.0.1:5173/

Swagger • http://127.0.0.1:8000/docs

Health • http://127.0.0.1:8000/api/health

Change Streams are in fallback mode on standalone local MongoDB; ordinary CRUD/read operations remain available.

Team responsibility map

Use these as the stated role allocation, but align contribution claims with actual work.

MEMBER 1

Naman Sharma
Register: 23BCE1683

Role: MongoDB Data Architect & Backend Lead

Focus: NoSQL model, collection design, embedding/reference strategy, seed/index design and database evidence.

MEMBER 2

Register: 23BCE1665

Role: AI & Evidence Analysis Lead

Focus: evidence workflow, scoring inputs, research/claim analysis and evaluation of the AI-assisted pipeline.

MEMBER 3

Register: 23BCE5048

Role: System Integration, Visualization & Evaluation Lead

Focus: integration, React views, graph/timeline presentation, testing and project demonstration evidence.

KEY TAKEAWAYS

VERITAS turns a debate into inspectable, persistent data.

01	**Structure**	Claims, evidence, counterarguments and conflicts are first-class records.
02	**Explain**	Scores expose their input factors; synthesis preserves uncertainty.
03	**Evolve**	Selected changes are persisted as temporal snapshots.
04	**Demonstrate**	MongoDB, FastAPI and React work together as a queryable system.

VERITAS organizes and explains the evidence landscape — it does not declare objective truth.

Thank you • Questions & discussion